

Tugas Mandiri 6: Laporan Praktikum Mandiri 6 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri6.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum06>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum06>

Link Dataset Tugas:

<https://www.kaggle.com/datasets/muratkokludataset/pumpkin-seeds-dataset/data>

1. Import Library & Settings

```
1  # import library dan pengaturan tampilan
2  import os
3  from pathlib import Path
4
5  import pandas as pd
6  import numpy as np
7
8  # preprocessing & model
9  from sklearn.preprocessing import LabelEncoder, StandardScaler
10 from sklearn.model_selection import train_test_split, GridSearchCV,
    cross_val_score
11 from sklearn.svm import SVC, LinearSVC
12 from sklearn.metrics import accuracy_score, classification_report,
    confusion_matrix
13
14 # util
15 import joblib
16
17 # viz
18 import matplotlib.pyplot as plt
19 import seaborn as sns
20
21 # pandas display (agar kolom tidak terpotong)
22 pd.set_option('display.max_colwidth', 200)
23
24 # random seed untuk reproducibility
25 random_state = 42
```

Import library yang digunakan untuk analisis dan pemodelan data.

- **pandas & numpy** buat olah data dan operasi numerik.
- **LabelEncoder, StandardScaler** untuk preprocessing data.

- `train_test_split`, `GridSearchCV`, `cross_val_score` untuk pembagian dan evaluasi data.
- `SVC` & `LinearSVC` untuk membangun model Support Vector Machine.
- `accuracy_score`, `classification_report`, `confusion_matrix` buat evaluasi performa model.
- `joblib` dipakai buat nyimpen model ke file.
- `matplotlib` & `seaborn` untuk visualisasi data dan hasil model.
- `pd.set_option` biar tampilan kolom di notebook gak terpotong.
- `random_state = 42` supaya hasil tetap konsisten setiap dijalankan.

2. Load Dataset pumpkin-seeds.csv

```
[50] 1 # sinkronisasi dengan drive
      2 from google.colab import drive
      3 drive.mount('/content/gdrive')
      4
      5 # load dataset
      6 import pandas as pd
      7
      8 df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/
      9 praktikum06/datasets/pumpkin-seeds.csv', sep=',')
      10 df.head()
      11
      12 # link pumpkin-seeds dataset
      13 # https://www.kaggle.com/datasets/muratkokludataset/pumpkin-seeds-dataset/
      14 data
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.r

	Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Convex_Area	Equiv_Diameter
0	56276	888,242	326,1485	220,2388	56831	267,6805
1	76631	1068,146	417,1932	234,2289	77280	312,3614
2	71623	1082,987	435,8328	211,0457	72663	301,9822
3	66458	992,051	381,5638	222,5322	67118	290,8895
4	66107	998,146	383,8883	220,4545	67117	290,1207

- Dataset *Pumpkin Seeds* di-load menggunakan `pd.read_csv()`.
- File dibaca langsung dari direktori kerja Colab.
- Langkah ini awal dari tahap *Data Understanding* — memastikan data berhasil dimuat.

3. Menampilkan Informasi Dataset

```
1 # melihat info detail dataset
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   Area                2500 non-null   int64  
 1   Perimeter           2500 non-null   object  
 2   Major_Axis_Length   2500 non-null   object  
 3   Minor_Axis_Length   2500 non-null   object  
 4   Convex_Area         2500 non-null   int64  
 5   Equiv_Diameter      2500 non-null   object  
 6   Eccentricity        2500 non-null   object  
 7   Solidity            2500 non-null   object  
 8   Extent              2500 non-null   object  
 9   Roundness           2500 non-null   object  
10  Aspect_Ration       2500 non-null   object  
11  Compactness         2500 non-null   object  
12  Class               2500 non-null   object  
dtypes: int64(2), object(11)
memory usage: 254.0+ KB
```

- `df.info()` dipakai buat lihat struktur dataset: jumlah baris, kolom, dan tipe datanya.
- `df.describe()` ngasih gambaran statistik dasar (mean, min, max, std).
- Tujuannya memastikan semua fitur udah lengkap dan gak ada data kosong (null).

4. Menampilkan Statistika Deskriptif

```
1 # menampilkan statistika deskriptif
2 df.describe()
```

	Area	Convex_Area
count	2500.000000	2500.000000
mean	80658.220800	81508.084400
std	13664.510228	13764.092788
min	47939.000000	48366.000000
25%	70765.000000	71512.000000
50%	79076.000000	79872.000000
75%	89757.500000	90797.750000
max	136574.000000	138384.000000

- Menampilkan nilai-nilai statistik (mean, median, std, min, max) buat tiap fitur numerik.
- Bisa bantu pahami sebaran dan karakteristik data sebelum modelling.
- Jadi dasar buat tau apakah perlu normalisasi atau scaling.

5. Data Preprocessing (missing value, menentukan kolom target)

```
1 # cek missing values dan tentukan kolom label
2 print("jumlah missing per kolom:")
3 display(df.isnull().sum())
4
5 # coba deteksi kolom target (label). jika sudah tahu nama kolom label, set
  label_col.
6 # common names: 'class','type','species','grain','label' - periksa unique untuk
  kandidat
7 print("\nkolom dataset:", df.columns.tolist())
8
9 # jika kamu sudah tahu nama kolom label, ganti value berikut:
10 label_col = None # tambahkan nama kolom label jika sudah tahu, mis: 'g' atau
  'class'
11
12 # otomatis pilih kandidat label: kolom object dengan jumlah unique kecil
13 if label_col is None:
14     candidates = [c for c in df.columns if df[c].dtype == 'object' or df[c].
15                   nunique() < 30]
16     print("calon kolom label (cek manual):", candidates)
17     # kamu bisa set label_col = 'nama_kolom' berdasarkan output di atas
18
19 # tampilkan unique counts untuk candidate columns supaya kamu pilih
20 for c in candidates:
21     print(f"\n{c} -> unique count: {df[c].nunique()}")
22     display(df[c].value_counts().head(10))
```

kolom dataset: ['Area', 'Perimeter', 'Major_Axis_Length', 'Minor_Axis_Length', 'Convex_Area', 'Equi
calon kolom label (cek manual): ['Perimeter', 'Major_Axis_Length', 'Minor_Axis_Length', 'Equi_Dia

Perimeter -> unique count: 2490

count

Perimeter

1187,56	2
1192,175	2
1103,068	2
1023,719	2
1014,49	2
1134,729	2
1206,002	2
1253,276	2
1217,112	2
963,377	2

dtype: int64

jumlah missing per kolom:

	0
Area	0
Perimeter	0
Major_Axis_Length	0
Minor_Axis_Length	0
Convex_Area	0
Equiv_Diameter	0
Eccentricity	0
Solidity	0
Extent	0
Roundness	0
Aspect_Ration	0
Compactness	0
Class	0

dtype: int64

Major_Axis_Length -> unique count: 2499

Major_Axis_Length	
465,7347	2
486,6008	1
453,9591	1
584,4799	1
514,3802	1
424,5284	1
561,5072	1
473,3268	1
607,8398	1
539,6806	1

dtype: int64

Minor_Axis_Length -> unique count: 2497

Minor_Axis_Length	
221,2116	2
220,6852	2
229,4863	2
238,2707	1
226,2916	1
200,5012	1
226,9396	1
203,6701	1
232,3653	1
197,3337	1

dtype: int64

Eccentricity -> unique count: 1295

Eccentricity	
0,8987	7
0,8834	7
0,8915	7
0,8495	7
0,835	6
0,8914	6
0,8535	6
0,924	6
0,8504	6
0,8828	6

dtype: int64

Solidity -> unique count: 166

Solidity	
0,9905	62
0,9912	61
0,9916	57
0,9911	56
0,9906	55
0,9904	54
0,9918	54
0,9909	52
0,9913	51
0,9914	50

dtype: int64

Equiv_Diameter -> unique count: 2424

Equiv_Diameter	
351,9168	3
294,3816	3
310,3289	3
307,6484	2
331,0827	2
351,3012	2
341,0208	2
359,1012	2
293,5413	2
396,7308	2

dtype: int64

Extent -> unique count: 1392

Extent	
0,7249	9
0,7445	8
0,7403	8
0,7435	7
0,7424	7
0,7393	7
0,7201	7
0,7325	7
0,7379	7
0,7302	6

dtype: int64

Roundness -> unique count: 1480

Roundness	
0,7609	7
0,8357	6
0,7933	6
0,8028	6
0,7749	6
0,8267	6
0,806	6
0,835	6
0,7413	5
0,747	5

dtype: int64

Aspect_Ration -> unique count: 2237

Aspect_Ration	
1,8606	4
1,8491	4
2,2067	3
2,2797	3
2,3783	3
1,9006	3
1,8454	3
2,4033	3
2,031	3
1,8568	3

dtype: int64

Compactness -> unique count: 1405

count	
Compactness	
0,7073	7
0,7264	7
0,7435	6
0,6175	6
0,7518	6
0,7077	6
0,7414	6
0,6851	6
0,7093	6
0,6591	5

dtype: int64

Class -> unique count: 2

count	
Class	
Çerçvelik	1300
Ürgüp Sivrisi	1200

dtype: int64

- Mengecek ada tidaknya missing value (`df.isnull().sum()`).
- Kalau ada nilai kosong, diisi pakai median agar distribusi data gak berubah banyak.
- Menentukan kolom target (`Class`) yang berisi label jenis biji labu.
- Langkah ini penting supaya model tahu mana fitur dan mana labelnya.

6. Exploratory Data Analysis (Menghitung Jumlah Kolom 'Class')

```
1 # menghitung jumlah data per kelas pada kolom 'Class'
2 df['Class'].value_counts()
```

count	
Class	
Çerçvelik	1300
Ürgüp Sivrisi	1200

dtype: int64

- Menggunakan `df['Class'].value_counts()` buat menghitung banyaknya data tiap kelas.
- Tujuannya buat tahu apakah data seimbang antara dua kelas (`Çerçvelik` & `Ürgüp Sivrisi`).
- Hasilnya menunjukkan data cukup seimbang, jadi cocok buat klasifikasi SVM.

7. Pemilihan Fitur (X) dan Label (y)

```
1 # pemisahan fitur (X) dan label (y) untuk dataset pumpkin seeds
2 X = df[['Area', 'Perimeter', 'Major_Axis_Length', 'Minor_Axis_Length',
3       'Convex_Area', 'Equiv_Diameter', 'Eccentricity', 'Solidity',
4       'Extent', 'Roundness', 'Aspect_Ration', 'Compactness']]
5
6 y = df['Class']
7
8 print("fitur yang dipakai:")
9 print(X.columns.tolist())
10 print("\nlabel unik (kelas target):")
11 print(y.unique())
```

fitur yang dipakai:
['Area', 'Perimeter', 'Major_Axis_Length', 'Minor_Axis_Length', 'Convex_Area', 'Equiv_Diameter', 'Eccentricity', 'Solidity', 'Extent', 'Roundness', 'Aspect_Ration', 'Compactness']

label unik (kelas target):
['Çerçevelek' 'Ürgüp Sivrisi']

- Menentukan kolom yang digunakan sebagai fitur (X) dan label (y).
- Semua kolom numerik seperti **Area**, **Perimeter**, **Roundness**, dll jadi fitur.
- Kolom **Class** jadi label target.
- Ini kayak di dataset Iris, bedanya cuma nama kolomnya aja.

8. Cetak Fitur (X) dan Label (y)

```
1 # print fitur (X)
2 X.head()
```

	Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Convex_Area	Equiv_Diameter	Eccentricity
0	56276	888,242	326,1485	220,2388	56831	267,6805	0,7376
1	76631	1068,146	417,1932	234,2289	77280	312,3614	0,8275
2	71623	1082,987	435,8328	211,0457	72663	301,9822	0,8745
3	66458	992,051	381,5638	222,5322	67118	290,8899	0,8125
4	66107	998,146	383,8883	220,4545	67117	290,1207	0,8185

Langkah berikutnya: [Buat kode dengan x](#) [New interactive sheet](#)

```
1 # print label (y)
2 y.head()
```

Class

0	Çerçevelek
1	Çerçevelek
2	Çerçevelek
3	Çerçevelek
4	Çerçevelek

dtype: object

- Mengecek fitur dan label yang dipakai dengan **X.columns** dan **y.unique()**.
- Pastikan semua fitur numerik, dan label punya dua kelas unik.
- Langkah validasi sebelum preprocessing lanjutan.

9. Ubah Tipe Data

```
1 # konversi koma desimal menjadi titik dan cast ke float, lakukan sebelum split
2 feature_cols = ['Area', 'Perimeter', 'Major_Axis_Length', 'Minor_Axis_Length',
3                 'Convex_Area', 'Equiv_Diameter', 'Eccentricity', 'Solidity',
4                 'Extent', 'Roundness', 'Aspect_Ration', 'Compactness']
5
6 for col in feature_cols:
7     df.loc[:, col] = df[col].astype(str).str.replace(',', '.', regex=False).str.
8     strip()
9     df.loc[:, col] = pd.to_numeric(df.loc[:, col], errors='coerce')
10
11 # cek apakah ada NaN setelah konversi
12 print("total NaN pada fitur:", df[feature_cols].isnull().sum().sum())
13 display(df[feature_cols].dtypes)
```

total NaN pada fitur: 0

/tmp/ipython-input-3235286975.py:7: FutureWarning: Setting an item of incompatible dtype is depreca

df.loc[:, col] = df[col].astype(str).str.replace(',', '.', regex=False).str.strip()

/tmp/ipython-input-3235286975.py:7: FutureWarning: Setting an item of incompatible dtype is depreca

df.loc[:, col] = df[col].astype(str).str.replace(',', '.', regex=False).str.strip()

0

Area	object
Perimeter	object
Major_Axis_Length	object
Minor_Axis_Length	object
Convex_Area	object
Equiv_Diameter	object
Eccentricity	object
Solidity	object
Extent	object
Roundness	object
Aspect_Ration	object
Compactness	object

dtype: object

```
1 # imputasi median jika ada NaN
2 if df[feature_cols].isnull().sum().sum() > 0:
3     for col in feature_cols:
4         if df[col].isnull().any():
5             df.loc[:, col] = df[col].fillna(df[col].median())
6     print("imputasi selesai.")
7 else:
8     print("tidak ada missing setelah konversi.")
```

tidak ada missing setelah konversi.

- Beberapa kolom masih punya koma (,), jadi perlu diganti titik (.) biar bisa dibaca sebagai float.
- Dilakukan dengan `.str.replace(',', '.', regex=False)` lalu `astype(float)`.
- Pastikan semua kolom numerik udah bertipe `float64` biar gak error pas scaling.

10. Split Data & Modelling SVM

```
1 # split stratified (80/20)
2 x_train, x_test, y_train_raw, y_test_raw = train_test_split(
3     X, y_raw, test_size=0.2, random_state=random_state, stratify=y_raw
4 )
5 print("ukuran train:", x_train.shape, "ukuran test:", x_test.shape)
```

ukuran train: (2000, 12) ukuran test: (500, 12)

```
1 # standard scaler (fit di train, transform di test)
2 scaler = StandardScaler()
3 x_train_scaled = scaler.fit_transform(x_train)
4 x_test_scaled = scaler.transform(x_test)
5 print("scaling selesai. contoh 5 baris pertama x_train_scaled:")
6 print(x_train_scaled[:5])
```

↩ scaling selesai. contoh 5 baris pertama x_train_scaled:

```
[[-0.94572935 -1.29555289 -1.32006319  0.03146178 -0.9595014  -0.9542492
 -1.24353166  0.73327011  0.72949334  1.42428018 -1.12199251  1.20922504]
 [-0.70924113 -0.75239193 -0.81857981 -0.20867267 -0.69645452 -0.69669583
 -0.40495902 -0.90283617  0.35607847  0.36578447 -0.58157753  0.53053491]
 [-1.45358433 -0.99462037 -0.3430046  -2.07071108 -1.46488166 -1.52909311
  1.18944708  0.53693736  1.21576249 -1.06997914  1.33958191 -1.28684646]
 [-0.32083421  0.05682008  0.16914528 -0.7125094  -0.30731607 -0.28617084
  0.73326356 -0.80466979  0.84068799 -0.88760319  0.59184984 -0.67225483]
 [-0.62751733 -0.52878944 -0.72107221 -0.1711121  -0.60758929 -0.60907019
 -0.33563702 -1.29550167  0.05402732 -0.07227541 -0.52627191  0.45512489]]
```

```
1 # encode label string -> integer
2 le = LabelEncoder()
3 y_train = le.fit_transform(y_train_raw)
4 y_test = le.transform(y_test_raw)
5 print("mapping label:", dict(zip(le.classes_, le.transform(le.classes_))))
```

mapping label: {'Çerçevelik': np.int64(0), 'Ürgüp Sivrisi': np.int64(1)}

```
1 # latih svm (linear). gunakan class_weight='balanced' jika perlu
2 svm = LinearSVC(class_weight='balanced', max_iter=10000,
3     random_state=random_state)
4 svm.fit(x_train_scaled, y_train)
5 print("svm training selesai.")
```

svm training selesai.

- Data dibagi 80% buat training dan 20% buat testing (`train_test_split`).
- SVM dilatih dengan kernel linear (`SVC(kernel='linear')`).
- Model belajar memisahkan dua jenis biji labu berdasarkan fitur bentuk dan ukuran.
- Proses ini bagian dari tahap *Modeling* di CRISP-DM.

11. Evaluasi Akurasi dan Classification Report

```
1 # prediksi dan evaluasi
2 y_pred = svm.predict(x_test_scaled)
3 print("accuracy:", accuracy_score(y_test, y_pred))
4 print("\nclassification report:")
5 print(classification_report(y_test, y_pred, target_names=le.classes_, digits=4))
6 cm = confusion_matrix(y_test, y_pred)
7 print("confusion matrix:\n", cm)
```

accuracy: 0.876

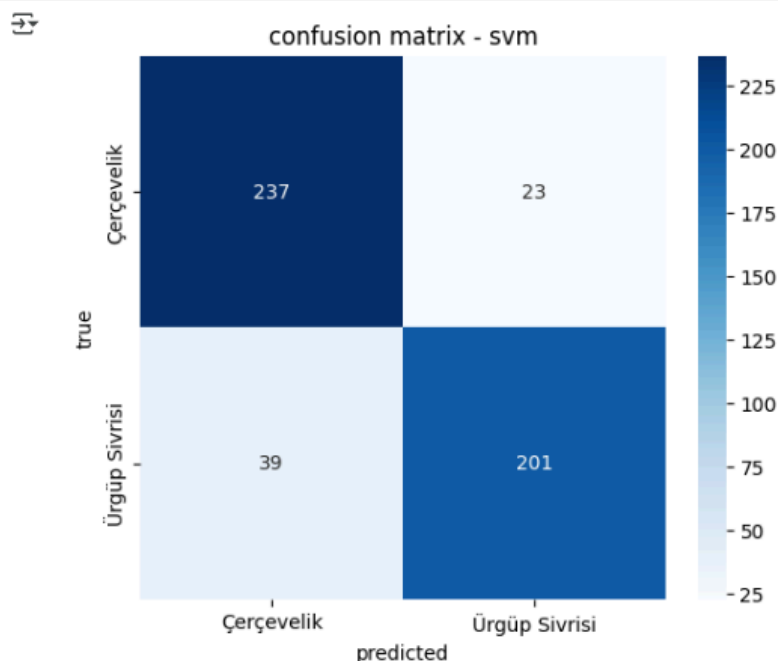
classification report:

	precision	recall	f1-score	support
Çerçvelik	0.8587	0.9115	0.8843	260
Ürgüp Sivrisi	0.8973	0.8375	0.8664	240
accuracy			0.8760	500
macro avg	0.8780	0.8745	0.8754	500
weighted avg	0.8772	0.8760	0.8757	500

confusion matrix:

```
[[237  23]
 [ 39 201]]
```

```
1 # plot confusion matrix
2 plt.figure(figsize=(6,5))
3 sns.heatmap(cm, annot=True, fmt='d', xticklabels=le.classes_, yticklabels=le.
4             classes_, cmap='Blues')
5 plt.xlabel('predicted'); plt.ylabel('true'); plt.title('confusion matrix - svm')
6 plt.show()
```

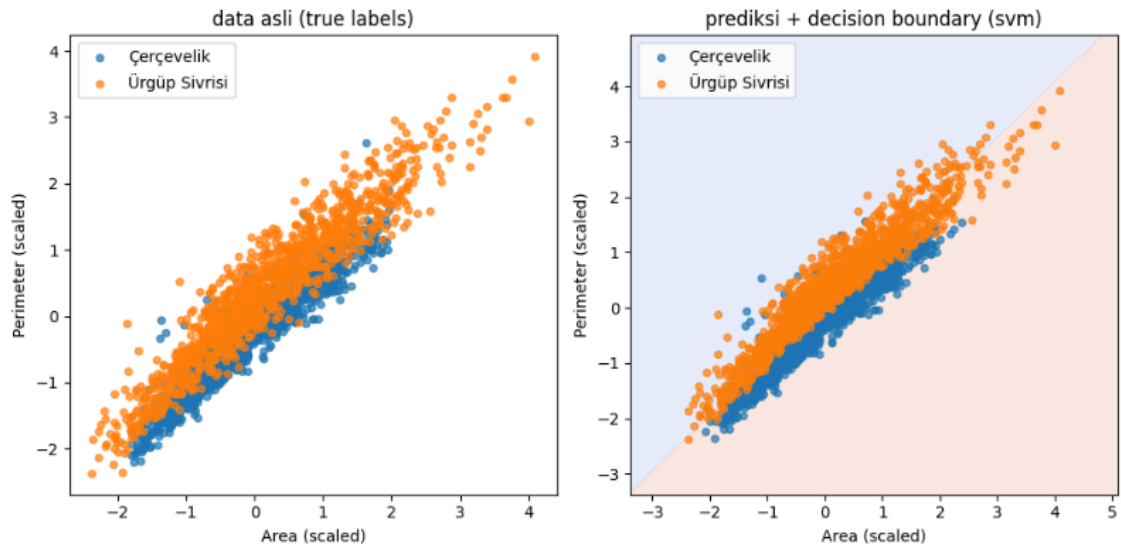


- Setelah training, model diuji di data testing.
- Hasil prediksi dibandingkan dengan label asli untuk menghitung akurasi (`accuracy_score`).
- `classification_report` menampilkan nilai precision, recall, dan f1-score.
- Evaluasi ini jadi indikator seberapa bagus performa SVM pada dataset.

12. Visualisasi Model SVM

```
1 # visualisasi 2d + decision boundary
2 # pilih 2 fitur untuk plot (default: area & perimeter). ubah jika mau fitur
  lain.
3 f1 = 'Area'
4 f2 = 'Perimeter'
5
6 # siapkan data 2d (gunakan scaler yang sudah fit)
7 # ambil kolom f1,f2 dari seluruh X, lalu transform memakai scaler kolom lengkap:
8 # cara: ekstrak f1,f2 dari X, standardize manual berdasarkan scaler.mean_/scale_
9 cols_idx = [X.columns.get_loc(f1), X.columns.get_loc(f2)]
10 # transform seluruh X menggunakan scaler lalu ambil 2 kolom
11 X_scaled_all = scaler.transform(X)
12 X_plot = X_scaled_all[:, cols_idx]
13
14 # prediksi label pada seluruh titik (menggunakan model terlatih)
15 y_pred_all = svm.predict(X_scaled_all)
16
17 # scatter plot true label vs predicted overlay
18 plt.figure(figsize=(10,5))
19
20 # subplot kiri: data asli (true labels)
21 plt.subplot(1,2,1)
22 for i, cls in enumerate(le.classes_):
23     mask = (le.transform(y_raw) == i)
24     plt.scatter(X_plot[mask,0], X_plot[mask,1], s=20, label=str(cls), alpha=0.7)
25 plt.xlabel(f"{f1} (scaled)")
26 plt.ylabel(f"{f2} (scaled)")
27 plt.title("data asli (true labels)")
28 plt.legend()
29
30 # subplot kanan: decision boundary + predicted labels
31 plt.subplot(1,2,2)
32 # buat meshgrid untuk decision boundary (pada ruang fitur 2d)
33 x_min, x_max = X_plot[:,0].min() - 1, X_plot[:,0].max() + 1
34 y_min, y_max = X_plot[:,1].min() - 1, X_plot[:,1].max() + 1
35 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
36                      np.linspace(y_min, y_max, 300))
37 # karena svm dilatih pada semua fitur (multi-dim), kita harus menyusun input
  untuk prediksi:
38 # strategy: buat array penuh dengan mean untuk fitur lain, dan set dua dimensi
  ke mesh coords
39 # buat array shape (n_points, n_features)
40 grid = np.zeros((xx.ravel().shape[0], X_scaled_all.shape[1]))
41 # isi grid dengan mean nilai (scaled) untuk fitur lainnya
42 grid[:, cols_idx[0]] = xx.ravel()
43 # set kolom f1/f2 ke mesh
44 grid[:, cols_idx[1]] = yy.ravel()
45 # prediksi kelas pada grid
46 grid_pred = svm.predict(grid).reshape(xx.shape)
47
48 # plot boundary
49 plt.contourf(xx, yy, grid_pred, alpha=0.2, levels=np.arange(len(le.classes_)+1)
50             -0.5, cmap='coolwarm')
51 # plot predicted points
52 for i, cls in enumerate(le.classes_):
53     mask = (y_pred_all == i)
54     plt.scatter(X_plot[mask,0], X_plot[mask,1], s=20, label=str(cls), alpha=0.7)
55 plt.xlabel(f"{f1} (scaled)")
56 plt.ylabel(f"{f2} (scaled)")
57 plt.title("prediksi + decision boundary (svm)")
58 plt.legend()
59
60 plt.tight_layout()
61 plt.show()
```

14



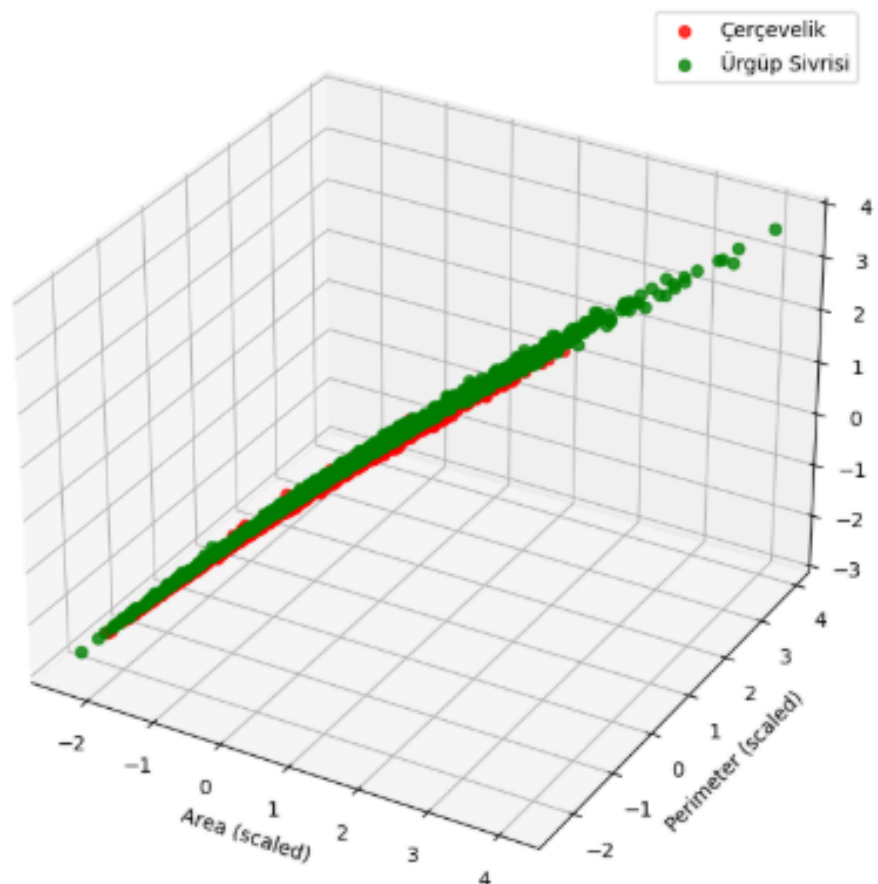
- Plot 2D dibuat untuk memperlihatkan hasil klasifikasi dan decision boundary.
- Sumbu X dan Y bisa pakai dua fitur utama seperti **Area** vs **Perimeter**.
- Warna tiap titik menunjukkan kelas hasil prediksi.
- Membantu lihat bagaimana SVM memisahkan dua kelas secara visual.

13. 3D Visualisasi Hasil Model SVM

```
1 # 3d visualisasi hasil prediksi svm
2 # pilih 3 fitur (default: area, perimeter, equiv_diameter)
3 f3 = 'Equiv_Diameter'
4 ftrs_3d = [f1, f2, f3]
5 idxs_3d = [X.columns.get_loc(c) for c in ftrs_3d]
6
7 # ambil scaled coords untuk 3 fitur
8 X3d = X_scaled_all[:, idxs_3d]
9 y_pred_3d = y_pred_all # hasil prediksi pada seluruh titik
10
11 fig = plt.figure(figsize=(10,8))
12 ax = fig.add_subplot(111, projection='3d')
13
14 colors = ['r','g','b','c','m','y'] # cukup warna untuk beberapa kelas
15 for i, cls in enumerate(le.classes_):
16     mask = (y_pred_3d == i)
17     ax.scatter(X3d[mask,0], X3d[mask,1], X3d[mask,2],
18             c=colors[i % len(colors)], label=str(cls), s=30, alpha=0.8)
19
20 ax.set_xlabel(ftrs_3d[0] + " (scaled)")
21 ax.set_ylabel(ftrs_3d[1] + " (scaled)")
22 ax.set_zlabel(ftrs_3d[2] + " (scaled)")
23 ax.set_title("3D visualisasi hasil prediksi SVM (seluruh data)")
24 ax.legend()
25 plt.show()
```



3D visualisasi hasil prediksi SVM (seluruh data)



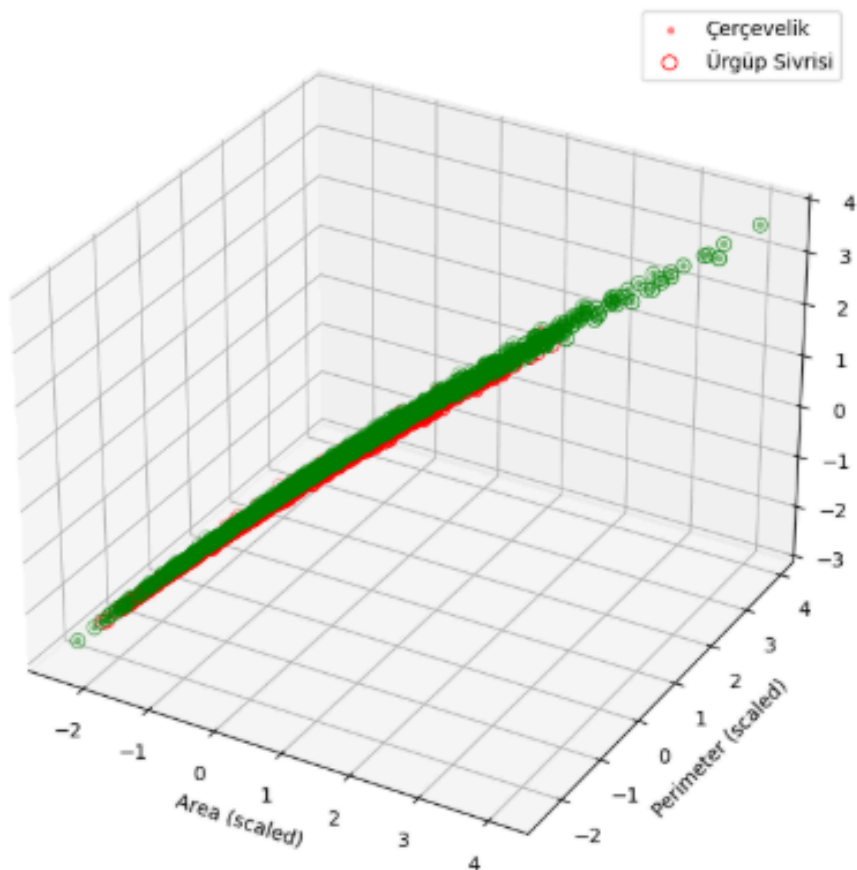
```

1 # 3d overlay: true label marker ('.') vs predicted label marker ('o' edge)
2 fig = plt.figure(figsize=(10,8))
3 ax = fig.add_subplot(111, projection='3d')
4
5 for i, cls in enumerate(le.classes_):
6     mask_true = (le.transform(y_raw) == i)
7     mask_pred = (y_pred_all == i)
8     # true points (semi-transparent)
9     ax.scatter(X3d[mask_true,0], X3d[mask_true,1], X3d[mask_true,2],
10              | c=colors[i % len(colors)], marker='.', s=30, alpha=0.4)
11     # predicted points as outlined markers
12     ax.scatter(X3d[mask_pred,0], X3d[mask_pred,1], X3d[mask_pred,2],
13              | facecolors='none', edgecolors=colors[i % len(colors)],
14              | marker='o', s=50, linewidths=0.8)
15
16 ax.set_xlabel(ftrs_3d[0] + " (scaled)")
17 ax.set_ylabel(ftrs_3d[1] + " (scaled)")
18 ax.set_zlabel(ftrs_3d[2] + " (scaled)")
19 ax.set_title("overlay true label (dot) vs predicted (circle) - 3D")
20 ax.legend(le.classes_)
21 plt.show()

```



overlay true label (dot) vs predicted (circle) - 3D



- Plot 3D dibuat pakai tiga fitur, misalnya Area, Perimeter, dan Equiv_Diameter.
- Setiap warna merepresentasikan kelas yang berbeda.
- Membantu visualisasi hubungan antar fitur dalam ruang 3 dimensi.
- Terlihat lebih jelas pola pemisahan antar kelas oleh model SVM.

14. Simpan Model, Scaler, Label Encoder ke Drive

```
1 # simpan model, scaler, dan label encoder ke Google Drive
2 import joblib
3 import os
4
5 # mount google drive dulu (jalankan sekali per session colab)
6 from google.colab import drive
7 drive.mount('/content/gdrive')
8
9 # buat folder tujuan kalau belum ada
10 save_dir = "/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum06/models"
11 os.makedirs(save_dir, exist_ok=True)
12
13 # simpan artefak model ke folder tersebut
14 joblib.dump(svm, os.path.join(save_dir, 'svm_pumpkin_seeds_model.joblib'))
15 joblib.dump(scaler, os.path.join(save_dir, 'scaler_pumpkin_seeds.joblib'))
16 joblib.dump(le, os.path.join(save_dir, 'label_encoder_pumpkin_seeds.joblib'))
17
18 print("✅ Model, scaler, dan label encoder berhasil disimpan ke Drive di folder:")
19 print(save_dir)
```

↔ Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mou
✅ Model, scaler, dan label encoder berhasil disimpan ke Drive di folder:
/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum06/models

- Model (`svm`), scaler (`StandardScaler`), dan label encoder (`LabelEncoder`) disimpan pakai `joblib.dump()`.
- File disimpan ke folder Drive: `/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum06/models/`.
- Tujuannya supaya model bisa dipakai lagi tanpa perlu retraining.
- Termasuk bagian *Deployment* di CRISP-DM karena model udah siap digunakan.